DevOps Explanation Request

Alright vicky 😈
Welcome to **DAY-23: Kubernetes RBAC — the part that decides who gets power and who gets "Forbidden" slapped on their face**.

I'll explain this **end-to-end**, exactly matching what you wrote, but **structured, crystal-clear, and production-accurate**.

☠️ Way (Without RBAC)

- Everyone authenticated → full access
- One wrong `kubectl delete` → cluster funeral ⚰️
- No separation between dev, ops, admin

So Kubernetes says:

> "Fine, I know who you are.
> Now tell me **what you're allowed to do**."

🛠️ How Kubernetes Solves This

Using **RBAC – Role Based Access Control**

RBAC answers **ONLY ONE QUESTION**:

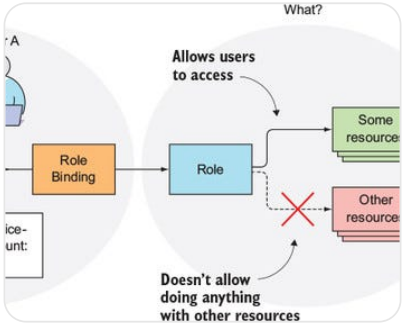
> **What can this identity do on which resource and where?**

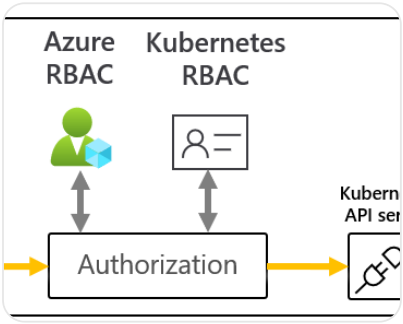
Step 1 – High-Level Overview

RBAC has **3 building blocks**:

```sql
Role        → WHAT actions are allowed
RoleBinding → WHO gets those actions
Namespace   → WHERE those actions apply
```

Kubernetes enforces RBAC **after authentication**.





Step 2 – Understand `kubectl auth can-i`

This is your **RBAC lie detector** 🕵️

```bash
kubectl auth can-i get pod
```

👉 Checks **current logged-in user**

Result:

```yaml
yes | no
```

Check as another user (impersonation)

```bash
kubectl auth can-i get pod --as adam
```

👉 Kubernetes pretends:

> "What if this request came from adam?"

⚠️ This does **NOT** log you in as adam
⚠️ This only tests RBAC rules

Who am I right now?

```bash
kubectl auth whoami
```

Example output:

```pgsql
Username: kubernetes-admin
Groups: [kubeadm:cluster-admins system:authenticated]
```

This confirms:

- **Authentication identity**
- **Groups attached to it**

Step 3 – Role (WHAT you can do)

A **Role** defines **permissions**, nothing more.

📄 `Role.yaml`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs:
  - get
  - list
  - watch
```

Meaning (line by line)

Field	Meaning
`apiGroups: [""]`	Core API (pods, services, configmaps)
`resources: ["pods"]`	Only pods
`verbs`	Allowed actions

👉 This Role **cannot delete**, **cannot update**, **cannot touch deployments**

Apply Role

```bash
kubectl apply -f Role.yaml
```

Role-related commands

```bash
kubectl get roles
kubectl get roles -A
kubectl describe role pod-reader
```

Count all Roles (interview favorite)

```bash
kubectl get roles -A --no-headers | wc -l
```

Step 4 – RoleBinding (WHO gets the role)

A Role **does nothing alone** ×

It must be **bound** to a user.

📄 `RoleBinding.yaml`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: adam
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Meaning

Section	Purpose
`subjects`	WHO gets permission
`roleRef`	WHICH role is granted

⚠️ `name: adam` **must exactly match** cert CN

Apply RoleBinding

```bash
kubectl apply -f RoleBinding.yaml
```

Verify

```bash
kubectl get rolebinding
kubectl describe rolebinding read-pods
```

Step 5 – Test RBAC (Without Logging In)

```bash
kubectl auth can-i get pod --as adam
# yes

kubectl auth can-i delete pod --as adam
# no
```

👉 RBAC working perfectly

But wait…

⚠️ Adam is **not logged in yet**

Step 6 – Login as Adam (kubeconfig)

RBAC alone is useless without **authentication**

So now we **add Adam to kubeconfig**

Add credentials

```bash
kubectl config set-credentials adam \
--client-key=../day-21/adam.key \
--client-certificate=../day-21/adam-final-certificate \
--embed-certs=true
```

What this does

- Adds Adam's **identity**
- Uses:
  - `adam.key` → PRIVATE KEY 🔐
  - `adam-final-certificate` → CERTIFICATE 📜

Create context

```bash
kubectl config set-context adam \
--cluster=kind-cka-cluster1 \
--user=adam
```

👉 Context = **user + cluster**

Switch context

```bash
kubectl config use-context adam
```

Verify login

```bash
kubectl auth whoami
```

Result:

```makefile
Username: adam
```

🎉 Adam is now logged in

Step 7 – Adam's Actual Permissions

Allowed

```bash
kubectl get pods
```

✅ Works

Forbidden

```bash
kubectl get deploy
```

× Error:

```sql
User "adam" cannot list resource "deployments"
```

Why?

- Role only allows **pods**
- Deployments belong to `apps` API group

RBAC doing its job 💪

Step 8 – Common Error: Certificate Expired

Sometimes this fails even if RBAC is correct.

Why?
👉 **Certificate expired**

Check cert validity:

```bash
openssl x509 -noout -dates -in ../day-21/adam-final-certificate
```

Example:

```ini
notBefore=Feb 2 08:46:24 2026
notAfter=Feb 3 08:46:24 2026
```

⚠️ Expired cert = authentication fails = RBAC never reached

Step 9 – Inspect kubeconfig

```bash
kubectl config view
```

Shows:

- Current context
- Current user
- Current cluster
- Embedded certs

This is your **single source of truth**

Step 10 – Common Mistakes (Very Real)

× Creating Role but no RoleBinding
× Wrong username in RoleBinding
× Assuming Role works cluster-wide
× Confusing Role vs ClusterRole
× Forgetting namespace scope
× Expired cert = RBAC “not working” illusion

🧠 Final Mental Model (Memorize This)

```scss
```

```
Authentication → Who are you?   (CERT)
Authorization  → What can you do? (RBAC)
```

RBAC logic:

```sql
Role        → Permissions
RoleBinding → Assignment
Namespace   → Scope
```

If you want next:

- Role vs ClusterRole (deep)
- Group-based RBAC
- system:masters danger zone
- Real CKA-style RBAC questions
- RBAC debugging checklist

Just say it 😏🔥